

Completed

- Built encounter-level features
- Aggregated labs, meds, conditions
- Created sepsis label from diagnosis text
- Prevented patient leakage with GroupShuffleSplit
- Used logistic regression with class balancing
- Evaluated with PR-AUC

Prompt 1:

I have the following pandas tables already loaded:

patients, encounters, conditions, observations, medications

This dataset follows a synthetic Mayo-style EHR schema where:

- patients.Id is the patient primary key
- encounters.Id is the encounter primary key
- encounters.PATIENT links to patients.Id
- conditions / observations / medications each contain:
 - PATIENT (FK to patients.Id)
 - ENCOUNTER (FK to encounters.Id)

Please generate a Python notebook cell that builds an ENCOUNTER-level modeling dataset for sepsis prediction.

Requirements:

1) Type normalization

- Convert all START / STOP / DATE / BIRTHDATE / DEATHDATE columns to pandas datetime (UTC), then strip timezone (tz-naive).
- Convert numeric fields (e.g., observations.VALUE, BASE_COST, TOTALCOST, PAYER_COVERAGE, DISPENSES) to float where present.
- Leave ongoing STOP dates as NaT/null (do NOT fill).

2) Patient cleanup

- Drop irrelevant patient columns if present: SUFFIX, MAIDEN, PASSPORT, DRIVERS, PREFIX, ADDRESS.
- Create IS_DECEASED flag from patients.DEATHDATE (1 if not null else 0).

3) Base table = encounters

- Start from encounters (1 row per encounter).
- Join patient demographics (BIRTHDATE, GENDER, RACE, ETHNICITY, IS_DECEASED) via encounters.PATIENT → patients.Id.
- Compute AGE_AT_ENCOUNTER in years using:

(encounters.START - patients.BIRTHDATE) / 365.25
(Do NOT use today or DEATHDATE for age.)

4) Aggregate encounter-level features:

Observations:

- Use numeric observations.VALUE only.
- Find the top N most frequent observation CODEs globally (set N=50).
- For those codes, compute mean/min/max VALUE per ENCOUNTER.
- Pivot to wide columns named:
 mean_OBS_<code>, min_OBS_<code>, max_OBS_<code>

Medications:

- Per ENCOUNTER:
 - count unique medication CODEs → MEDICATION_COUNT
 - sum all available cost columns (TOTALCOST / BASE_COST / etc.) → TOTAL_MED_COST

Conditions (optional but preferred):

- Per ENCOUNTER: count unique condition CODEs → CONDITION_COUNT

5) Sepsis label (encounter-level, no leakage):

- Create SEPSIS_LABEL = 1 if any conditions.DESCRPTION for that same ENCOUNTER contains
 "sepsis" or "septic" (case-insensitive), else 0.
- Do NOT use patient-level "ever had sepsis".

6) Output

- Merge all encounter-level aggregates into a single dataframe keyed by encounter Id.
- Name final dataframe `encounter\_features`.
- Print:
 - encounter_features.shape
 - SEPSIS_LABEL prevalence (mean)
- Display encounter_features.head().

Include clear comments explaining each step.

Write defensively (check columns exist, handle empty tables gracefully).

Code changes I made from prompt output:

- Observation: It was pivoting the whole dataframe (obs_pivot = obs_agg.pivot(index="ENCOUNTER", columns="CODE"))
 - use
 .set_index(["ENCOUNTER", "CODE"])[["mean", "min", "max"]].unstack("CODE")
- Medication costs: It was summing COVERAGE into "cost"

- You include both "COST" and "COVERAGE" columns in ROW_MED_COST. That mixes:
 - what was charged
 - what payer covered
- For a first-pass, I'd keep:
 - TOTALCOST (or sum of cost columns)
 - PAYER_COVERAGE separately
- Potential leakage: conditions used both as feature and label
 - CONDITION_COUNT as a feature and also use conditions to create the label. -> keep CONDITION_COUNT but exclude sepsis-related diagnoses

Prompt 2:

I already built `encounter\_features` (61k rows, encounter-level) and it already includes:

- AGE_AT_ENCOUNTER
- mean_OBS_*, min_OBS_*, max_OBS_* for top 50 obs codes
- MEDICATION_COUNT, TOTAL_MED_COST, TOTAL_MED_PAYER_COVERAGE, TOTAL_MED_OOP_EST
- CONDITION_COUNT (excluding sepsis-related diagnoses)
- SEPSIS_LABEL (0/1)

I have also already dropped identifier columns into `encounter\_features\_model` (or can do so in this cell).

Please generate a Python notebook cell that does ONLY the following (do NOT redo joins/aggregations/labeling):

1) Create X and y

- y = SEPSIS_LABEL
- X = all remaining columns after dropping SEPSIS_LABEL
- Ensure we do not include identifier columns even if they still exist (Id, PATIENT, PATIENT_ID, PROVIDER, ORGANIZATION, PAYER)

2) Encode categoricals

- One-hot encode GENDER, RACE, ETHNICITY if present using `pd.get_dummies(drop_first=True)`

3) Handle missingness WITHOUT pretending missing labs are zero

- Replace +/-inf with NaN
- For true "count/cost" columns (MEDICATION_COUNT, TOTAL_MED_COST, TOTAL_MED_PAYER_COVERAGE, TOTAL_MED_OOP_EST, CONDITION_COUNT), fill NaN with 0
- For all remaining numeric columns (mostly labs/vitals), impute NaN with the column median

- Optionally add missingness indicators for lab columns only: <col>__missing

4) Train/evaluate a fast baseline model suitable for extreme class imbalance

- Use a stratified train/test split
- Train LogisticRegression (or SGDClassifier with log loss) with class_weight="balanced"
- Report ROC-AUC and PR-AUC (average precision)
- Also print: number of positives in train and test

5) Show the top 15 absolute coefficient features (if LogisticRegression) for a sanity check.

Keep code robust and commented.

Code changes I made from prompt output:

- Big leakage concern: since encounters are split randomly, if the same patient appears in train & test, you can get optimistic performance (because patient specific baseline leaks)
 - [GroupShuffleSplit by Patient](#)
- Reasoncode & CODE are categorical but they're being treated as numeric. Drop for baseline model.
- Lab values + costs + counts are on wildly different scales. Logistic regression *can* handle it, but with saga + imbalance + sparse flags, scaling helps convergence a lot.
 - StandardScaler